

Name :- Rucha Nitin Pawar

Branch :- AI&DS

Div :- B

Roll No:- 70

Practical Batch :- B4

Aim:- Data Ingestion and cleaning :Acquire data and prepare it for analysis.

Theory

1. i) What is data ingestion and why is it important in data science?

Data ingestion is the process of collecting and importing data from different sources such as databases, files, APIs, or streaming systems into a system where it can be stored and analyzed. It is important in data science because raw data often comes from multiple sources and in different formats, so ingestion ensures the data is gathered, organized, and made ready for analysis, which is the foundation for accurate insights and decision-making.

ii) which python library is commonly used for data ingestion and cleaning?

A commonly used Python library for data ingestion and cleaning is pandas, which provides powerful tools to read data from various formats like CSV, Excel, and SQL databases, and offers functions to clean, transform, and manipulate data efficiently.

2. Why is data cleaning necessary before performing analysis or machine learning?

Data cleaning is necessary before analysis or machine learning because real-world data is often incomplete, inconsistent, or contains errors such as missing values, duplicates, and incorrect formats. If such data is used directly, it can lead to inaccurate results, biased models, and poor predictions. Cleaning the data ensures it is consistent, reliable, and properly structured, which improves the quality of analysis and helps machine learning models perform more accurately and efficiently.

3. How do load a csv file into a pandas dataframe?

You can load a CSV file into a pandas DataFrame using the `read_csv()` function from the pandas library. First, you import pandas, then pass the file path of the CSV file to this function, which reads the data and stores it in a DataFrame for further analysis. For example:

```
import pandas as pd
df = pd.read_csv("file.csv")
print(df.head())
```

4. What is difference between head() and tail() functions in pandas?

In pandas, both `head()` and `tail()` functions are used to quickly view data in a DataFrame, but they differ in which part of the data they show. The `head()` function returns the first few rows of the dataset (by default, the first 5 rows), which is useful for inspecting the beginning of the data and checking column structure. On the other hand, the `tail()` function returns the last few rows (by default, the last 5 rows), which helps you examine the end of the dataset and verify how the data concludes.

5. How can you identify missing values in dataset?

You can identify missing values in a dataset in pandas by using functions like `isnull()` or `isna()`, which return a boolean result indicating where values are missing. To get a summary, you can combine them with `sum()` to count missing values in each column. For example, `df.isnull()` shows all missing positions, and `df.isnull().sum()` gives the total number of missing values per column, helping you quickly understand where data is incomplete.

```
from google.colab import files
uploaded = files.upload()
print(uploaded)
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving employee_performance_data.csv to employee_performance_data.csv
{'employee_performance_data.csv': b'Employee_ID,Name,Age,Gender,Department,Join_

```
import pandas as pd
df = pd.read_csv("employee_performance_data.csv")
print("Dataset loaded successfully!")
print(df.head())
```

Dataset loaded successfully!

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0	
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0	
2	103	Charlie	29	Female	IT	2007-02-15	NaN	
3	104	David	42	Male	Marketing	2015-12-29	31016.0	

4	105	Eve	40	Female	IT	2005-02-04	119789.0
---	-----	-----	----	--------	----	------------	----------

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
0	13.0	2.0	9488.0	44.0
1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

```

numeric_cols = df.select_dtypes(include=['number']).columns

mean_filled = df.copy()
median_filled = df.copy()

for col in numeric_cols:
    mean_filled[col] = mean_filled[col].fillna(mean_filled[col].mean())
    median_filled[col] = median_filled[col].fillna(median_filled[col].median())

zero_filled = df.fillna(0)

print("Mean Strategy (numeric columns only):")
print(mean_filled.head())

print("\nMedian Strategy (numeric columns only):")
print(median_filled.head())

print("\nZero Strategy (all columns filled with 0):")
print(zero_filled.head())

```

Mean Strategy (numeric columns only):

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
0	101	Alice	50	Female	Marketing	2009-01-18	62606.000000	
1	102	Bob	36	Female	Marketing	2012-01-01	41534.000000	
2	103	Charlie	29	Female	IT	2007-02-15	76338.368421	
3	104	David	42	Male	Marketing	2015-12-29	31016.000000	
4	105	Eve	40	Female	IT	2005-02-04	119789.000000	

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
0	13.0	2.0	9488.0	44.0
1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

Median Strategy (numeric columns only):

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0	
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0	
2	103	Charlie	29	Female	IT	2007-02-15	69099.0	
3	104	David	42	Male	Marketing	2015-12-29	31016.0	
4	105	Eve	40	Female	IT	2005-02-04	119789.0	

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
0	13.0	2.0	9488.0	44.0

1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

Zero Strategy (all columns filled with 0):

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary \
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0
2	103	Charlie	29	Female	IT	2007-02-15	0.0
3	104	David	42	Male	Marketing	2015-12-29	31016.0
4	105	Eve	40	Female	IT	2005-02-04	119789.0

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
0	13.0	2.0	9488.0	44.0
1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

```
import pandas as pd
# Re-loading df to ensure it's defined, in case previous cell wasn't run
df = pd.read_csv("employee_performance_data.csv")

categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col] = df[col].astype(str).str.lower().str.strip()

print("Cleaned categorical columns (head of the DataFrame after processing):")
print(df[categorical_cols].head())
```

Cleaned categorical columns (head of the DataFrame after processing):

	Name	Gender	Department	Join_Date
0	alice	female	marketing	2009-01-18
1	bob	female	marketing	2012-01-01
2	charlie	female	it	2007-02-15
3	david	male	marketing	2015-12-29
4	eve	female	it	2005-02-04

```
df = df.fillna(df.mean(numeric_only=True))
print("Missing numeric values filled")
```

Missing numeric values filled

```

numeric_cols = df.select_dtypes(include='number').columns
for col in numeric_cols :
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    Outliers = df[ (df[col] < (Q1 - 1.5*IQR)) | (df[col] > (Q3 +1.5*IQR))]
    print(f"\nOutliers in {col}: {len(Outliers)}")

```

Outliers in Employee_ID: 0

Outliers in Age: 0

Outliers in Salary: 0

Outliers in Experience: 0

Outliers in Performance_Rating: 0

Outliers in Bonus: 1

Outliers in Work_Hours_Per_Week: 0

```

def clean_dataset(data):
    data = data.drop_duplicates()
    data.columns = data.columns.str.lower().str.strip()
    data = data.fillna(data.mean(numeric_only=True))
    return data

```

```

cleaned_df = clean_dataset(df)
print("Automated cleaned Data :")
print(cleaned_df.head())

```

Automated cleaned Data :

	employee_id	name	age	gender	department	join_date	salary \
0	101	alice	50	female	marketing	2009-01-18	62606.000000
1	102	bob	36	female	marketing	2012-01-01	41534.000000
2	103	charlie	29	female	it	2007-02-15	76338.368421
3	104	david	42	male	marketing	2015-12-29	31016.000000
4	105	eve	40	female	it	2005-02-04	119789.000000

	experience	performance_rating	bonus	work_hours_per_week
0	13.0	2.0	9488.0	44.0
1	1.0	1.0	5206.0	40.0
2	9.0	4.0	10134.0	33.0
3	7.0	1.0	10977.0	42.0
4	9.0	4.0	12721.0	36.0

```

report = {
    "Total Rows Before Cleaning": len(df),
    "Missing values After Cleaning": cleaned_df.isnull().sum().to_dict(),
    "Final Row Count ": len(cleaned_df)
}

```

```
}  
print("Data Cleaning Report:")  
for key, value in report.items():  
    print(f"{key}: {value}")
```

```
Data Cleaning Report:  
Total Rows Before Cleaning: 20  
Missing values After Cleaning: {'employee_id': 0, 'name': 0, 'age': 0, 'gender':  
Final Row Count : 20
```

```
report = {  
    "Total Rows Before Cleaning": len(df),  
    "Missing values After Cleaning": cleaned_df.isnull().sum().to_dict(),  
    "Final Row Count ": len(cleaned_df)  
}  
print("Data Cleaning Report:")  
for key, value in report.items():  
    print(f"{key}: {value}")
```

```
Data Cleaning Report:  
Total Rows Before Cleaning: 20  
Missing values After Cleaning: {'employee_id': 0, 'name': 0, 'age': 0, 'gender':  
Final Row Count : 20
```

Conclusion:

Data ingestion and cleaning are essential first steps in the data analysis process. By collecting data from reliable sources and systematically cleaning it—removing duplicates, handling missing values, correcting inconsistencies, and standardizing formats—we ensure the dataset is accurate and usable. Tools like pandas make this process efficient and scalable. Properly prepared data reduces errors, improves analysis quality, and leads to more trustworthy insights and decisions.